

VisCanvas: A Node-Based Interface for Exploratory Visualization Authoring with LLMs

Yuki Ueno 

Bretho Danzy III 

Zhuojun Jiang 

Chris Bryan 

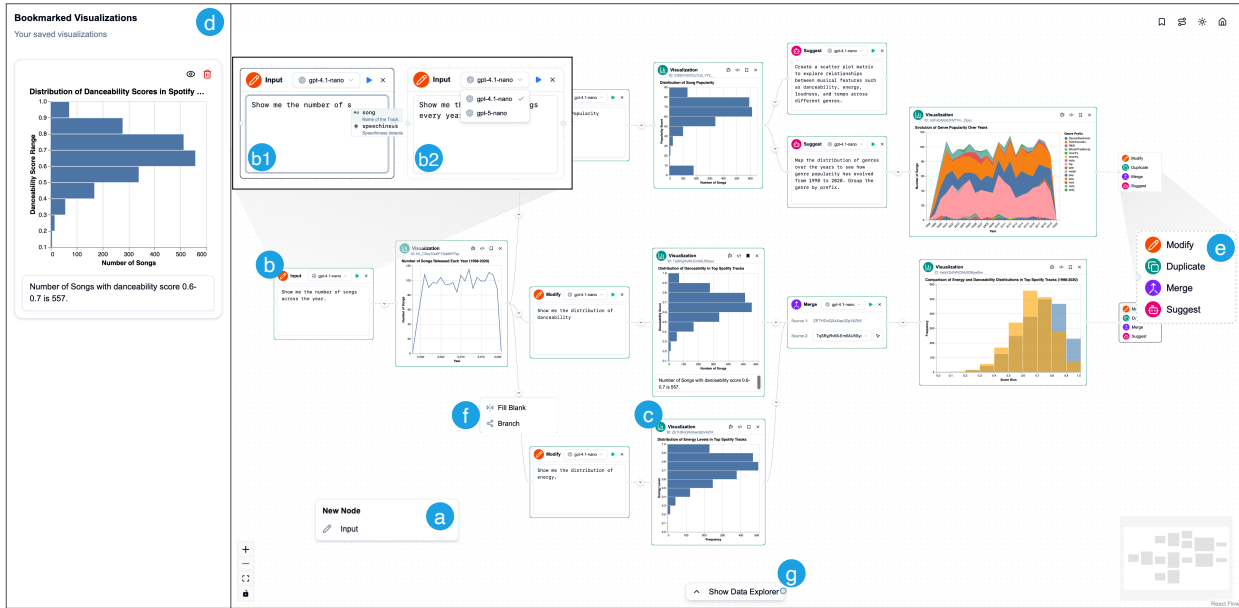


Figure 1: VISCANVAS supports exploratory visualization authoring through a node-based interface comprising: (a) a Context Menu for adding Text Input Nodes; (b) a Text Input Node for generating and updating visualizations, with (b1) a column-name autocompletion feature and (b2) a model selector; (c) a Visualization Node for viewing, interpreting, editing, and bookmarking results; (d) a Bookmark Panel for displaying bookmarked visualizations; (e) a Menu Node for creating new analysis paths via Modify, Duplicate, Merge, and Suggest operations; (f) an edge dropdown menu for extending connections via Fill Blank and Branch operations; and (g) a Data Explorer for inspecting datasets.

Abstract—Visual data analysis involves both open-ended exploration and targeted question answering. Visualization authoring tools support this process by enabling users to create visualizations for these tasks. With the rise of large language models (LLMs), substantial effort has been devoted to developing visualization authoring tools that use natural language instructions. However, existing systems are typically based on a linear chat interface, which is not well suited to exploratory visual analysis workflows. In this paper, we introduce VisCanvas, a node-based interface for exploratory visualization authoring with LLMs. VisCanvas allows users to create, revise, branch, and merge visualizations in a non-linear way, enabling more efficient exploration of multiple analytical directions. We conducted a user study with 20 participants to evaluate the effectiveness of VisCanvas compared to a baseline chat-based interface. The results show that VisCanvas facilitates more diverse data interaction while maintaining performance levels (i.e., cognitive load and usability) that are indistinguishable from current prevailing methods. We then distill design principles for future AI-assisted visualization authoring environments. All supplemental materials required to reproduce the study are available at <https://osf.io/gsxhn>.

Index Terms—Visualization Authoring, Large Language Models, Exploratory Analysis, Provenance

1 INTRODUCTION

Today's proliferation of data has amplified the need for accessible visualization tools that lower the barrier to creating effective data representations [40]. Such tools span levels of abstraction, ranging from formal grammars [8, 31, 41, 42, 55] to graphical interfaces [1, 30, 35, 38, 39]. With recent advances in large language models (LLMs), researchers have begun exploring visualization authoring through natural-language

instructions [15, 22, 33, 48]. In these systems, users can iteratively define and refine visualizations by providing instructions through a chat or spoken-language interface in a conversational manner.

However, visual analysis is often inherently exploratory, as it involves both constructing visualizations and discovering knowledge through them. For analysis-oriented tasks, exploration is driven by the need to uncover insights from data, ranging from answering targeted questions to identifying questions of interest through open-ended exploration [9]. In contrast, for design-oriented or storytelling contexts, the focus can center on the visual representation itself, such as comparing design alternatives and multiple representations to converge on an effective and expressive visualization [17].

While authoring systems lower the barriers to creating visualizations, they often assume linear workflows [40] or depth-first exploration strategies [57], and chat-based interaction paradigms further reinforce this linearity. This mismatch can become especially problematic during

• Yuki Ueno, Bretho Danzy III, Zhuojun Jiang, and Chris Bryan are with Arizona State University. E-mail: {yueno, bdanz, zjian115, cbryan16}@asu.edu

Manuscript received xx xxx. 202x; accepted xx xxx. 202x. Date of Publication xx xxx. 202x; date of current version xx xxx. 202x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org. Digital Object Identifier: xx.xxxx/TVCG.202x.xxxxxx

early-stage, analysis-oriented authoring, where users need to browse data, branch from partial findings, compare alternatives, and pursue multiple hypotheses. However, chat-based systems force users to externalize this exploratory reasoning through a single sequential interaction thread. As a result, it can become difficult to manage divergent analytical directions, compare alternatives side-by-side, and revisit the rationale behind prior decisions.

This motivates the present work, which investigates how to support analysis-oriented exploratory visualization authoring within a human-LLM collaboration context. To this end, we introduce VISCANVAS, an AI-assisted visualization authoring system that adopts a node-based interaction paradigm. In VISCANVAS, each node represents a visualization instance together with its corresponding Vega-Lite [41] specification, allowing users to explicitly branch, compare, revise, and merge alternative analytical directions. By externalizing the exploration space as a node graph, VISCANVAS enables users to maintain multiple threads or branches of analysis and exploration simultaneously and reason about analytical alternatives in parallel, rather than through a single linear conversational thread.

VISCANVAS is primarily positioned for early-stage analytical exploration and chart construction, where users iteratively generate, branch, compare, and refine Vega-Lite visualizations. Each visualization remains a reusable Vega-Lite specification, users can further refine encodings, labels, styling, and presentation details through specification editing or downstream authoring workflows. It does this by supporting non-linear exploration during the phase in which users are still identifying promising questions, patterns, and analytical directions, and asking questions about the underlying dataset. To enable this, VISCANVAS combines a graph-centric frontend with a staged LLM pipeline that translates user instructions into analysis goals and Vega-Lite specifications while using self-reflection to detect and repair invalid outputs. The interface is designed around a graph-focused UI and UX to support chat-based authoring together with branching, merging, on-demand suggestions, and semantic zooming, allowing users to move fluidly between focused refinement and breadth-oriented exploration on an unlimited canvas. In contrast to prior chat-based visualization tools that primarily reinforce linear workflows, VISCANVAS represents, to our knowledge, an early investigation into chat-based visualization authoring in an exploratory, non-linear manner.

We evaluate VISCANVAS through a controlled user study with 20 students. The results indicate that VISCANVAS promotes more exploratory authoring behaviors, supporting breadth-first exploration while maintaining a cognitive burden comparable to that of conversational interfaces (e.g., ChatGPT-like systems for visualization authoring). Participants also preferred VISCANVAS for open-ended exploratory scenarios because it better supported organizing thoughts, navigating the exploration space, parallel discovery, and comparing and merging visualizations to generate new insights.

In summary, this paper makes the following contributions:

- We develop and study a node-based interaction paradigm that externalizes multi-strategy analytical reasoning beyond linear LLM conversations through operations such as branching, comparison, and merging.
- We present VISCANVAS, a node-based system for exploratory visualization authoring with LLMs. To support reproducibility, the tool’s codebase is available at <https://github.com/svl-at-asu/VisCanvas>.
- We provide empirical evidence from a controlled user study (N=20) showing how node-based LLM interfaces shape exploratory authoring behaviors (study materials are available at <https://osf.io/gsxhn>). We also reflect on implications for future AI-assisted authoring environments.

2 RELATED WORK

2.1 Visualization Authoring Tools

A variety of authoring approaches facilitate visualization creation. For instance, toolkits like D3.js [8] and Vega [42] offer full expressive

power but require substantial programming expertise. Other declarative systems, inspired by the *Grammar of Graphics* [56], including Vega-Lite [41], Gosling [31], and ggplot2 [55], allow authors to easily specify a wide variety of visualizations using concise and uniform languages. In addition to programming-based approaches, a range of interactive visualization authoring systems have been developed [40]. For example, Tableau [1], Power BI [4], and Polaris [46] enable users to construct visualizations by mapping data fields to visual encoding channels, rather than relying on predefined templates [3, 5]. Furthermore, computational notebooks such as Jupyter Notebook [28] and Colab [2] are common tools for seamlessly combining text, code, and visualizations in a document, which are typically presented in a linear manner [52].

These tools allow authors to create visualizations efficiently, but they are generally well-suited to depth-first exploration strategies, in which users need to provide detailed specifications, hindering rapid exploration of multiple alternative analytical directions. As a result, broad exploration across such directions remains an open problem [57]. Specifically, Voyager [57] and Voyager 2 [58] are particularly important examples of systems that support exploratory data analysis through visualization recommendation and faceted browsing.

While sharing similar goals of lowering authoring barriers, VISCANVAS differs in both scope and interaction model. In particular, compared with Voyager systems, VISCANVAS does not primarily support exploration through browsing system-recommended views over the data space. Instead, it focuses on human-LLM co-authoring, where users iteratively create, revise, branch, and combine visualizations. Our work focuses on early-stage, analysis-oriented exploratory authoring, where users are still forming questions and comparing possible analytical directions. To support this phase, VISCANVAS introduces a node-based interface that facilitates branching, comparison, and reuse of intermediate visualization states in human-LLM collaboration.

2.2 Using LLMs for Visualization Construction

Natural language interfaces have been extensively explored across various stages of visualization workflows [44]. Recent advances in LLMs further enable the translation of textual descriptions into specifications or code [25, 51]. Systems such as LIDA [15], ChartLlama [22], Chat2VIS [33], PlotGen [19], and CoDA [14] leverage LLMs to generate Python code for data visualization. Other systems, including DynaVis [50], ChartGPT [48], and VisPilot [53], generate Vega-Lite specifications; meanwhile InterChat [13] produces D3.js code through interaction-augmented instructions [43]. More closely related to exploratory visual analysis, AI Threads [24] is an LLM-based prototype that combines a Main Chat and Thread panel to support branching dialogue. While these systems lower the barrier to entry, they largely rely on linear, chat-based human-LLM interaction, where users prompt the LLM and the visualization is iteratively updated. Such interaction models can be seen as aligning with linear workflow assumptions [40] or depth-first exploration strategies [57], similar to those found in many visualization authoring tools. This linear workflow is particularly limiting for analysis-oriented exploratory authoring, in which users often need to branch from intermediate results, compare alternative views, and revisit partially developed ideas. Without explicit structural mechanisms for relating these alternatives, users struggle to maintain a mental model of the evolving authoring process.

To address the limitations of linear interaction histories, some systems have explored recording user interactions and LLM responses graphically [23]. VisPilot [53] integrates an Authoring Flow that records text/visual prompts and LLM responses as a chronological process view, with generated visualizations shown to help users review how an authoring task was completed; however, these recorded states primarily support retrospective review rather than reuse as actionable starting points for forking or recombining analysis. Urbanite [34] goes further by using LLM-assisted executable dataflows for urban visual analytics, where heterogeneous nodes encode data loading, transformation, and visualization modules connected by data or interaction dependencies. This provides strong support for constructing and inspecting analytics pipelines, but its exploration is centered on workflow

modules and dataflow versions; users typically explore alternatives by modifying pipelines rather than directly manipulating lightweight candidate charts. VISCANVAS addresses this gap by making generated visualization states the primary exploration units: each Visualization Node is an actionable Vega-Lite state that users can revisit, revise, duplicate, branch from, compare, and merge during early-stage analysis.

2.3 Provenance for Visualization Creation

Extensive visualization research has been devoted to supporting provenance [37, 59]. Ragan et al. [37] surveyed provenance systems and characterized prior work according to the types of provenance information captured and their purposes. Similarly, Xu et al. [59] reviewed provenance research using a Why-What-How framework. Prior studies have shown that provenance information supports users in reviewing past actions, comparing alternatives, and reconstructing analytic processes, thereby facilitating sensemaking, discovery, and creative reasoning [45, 59]. Accordingly, provenance systems have emphasized retrospective understanding and explanation of analysis workflows.

From the perspective of visualization creation, many systems record intermediate visual outputs produced during analysis [37]. A number of visual analytics tools (e.g., [11, 16, 18, 26, 32, 49, 60]) employ graphical histories [23] to represent the evolution of visualization states, enabling users to save, revisit, and review prior visualizations. In particular, systems such as Image Graphs [32], GraphTrail [18], ExPlates [26], VisFlow [60], and sMolBoxes [49] adopt graph-based interfaces in which each node represents a visualization state. Building on this line of work, FlowSense [61] integrates natural language processing into VisFlow, allowing users to construct and modify visualizations through conversational input. While natural language interfaces reduce learning overhead, they still require users to specify precise computational instructions due to limitations in language understanding and inference.

More recently, provenance-inspired designs have been explored in the HCI community to help users organize ideas and iteratively refine instructions when working with generative models [6, 21, 27, 29, 47]. These systems demonstrate the value of representing generative outputs as nodes, enabling users to reflect on, compare, and manage multiple ideas. For instance, Spellburst [6] closely aligns with our interaction model by representing LLM-generated artifacts as nodes that users can branch and merge. Unlike Spellburst, which focuses on creative-coding variants, VISCANVAS treats nodes as data-grounded Vega-Lite visualization states tied to analysis goals, enabling visualization-specific suggestions, comparison, and merging.

Our work extends this line of research to AI-supported visualization authoring, where node-based provenance structures must support not only idea management but also data-grounded analytical exploration. By foregrounding prior visualization states as starting points for further exploration, our work advances provenance and authoring-focused research toward analysis-oriented exploratory visualization authoring.

3 DESIGN CONSIDERATIONS

To develop a design that achieves our goal of analysis-oriented exploratory visualization authoring within a human-LLM collaborative context, we present a set of design considerations. These were distilled from a review and synthesis of prior work in AI-assisted data analysis [20], visualization authoring with LLMs [48, 50], exploratory search and analysis [54, 57], and provenance-inspired design [6, 29, 36, 59].

C1: Exploration structure at multiple levels of granularity. In analysis-oriented visualization authoring, users often pursue multiple partial hypotheses, revisit prior results, and compare alternative directions. Explicitly showing the structure of these states (including their branching) can support overview and recall [37, 59], however linear authoring tools are constrained to only a sequential view of such history. At the same time, prior work such as StoryFacets shows that branching history views can become overwhelming as analyses grow [36]. One potential solution is *providing explicit views of the exploration structure at multiple levels of granularity (i.e., both local and global detail levels) that emphasize the exploratory and branching structure of analysis while allowing users to focus on level-of-detail specific information.*

C2: On demand recommendations to assist in data exploration.

Exploratory visual analysis requires users not only to refine existing ideas but also to ideate and explore promising next steps. Prior work such as Voyager showed that visualization recommendations can effectively support breadth-oriented exploration by helping users survey possible directions and encounter views they might not have specified on their own [57]. At the same time, such suggestions can be detrimental if they impose too much cognitive overhead, are excessive, poorly timed, or remove the user's locus of control [62]. Therefore, *suggestions should be lightweight, context-aware, and on demand (i.e., triggered by users).*

C3: Merging and branching of ideas. As mentioned for C1, analysis-oriented authoring rarely proceeds as a sequential chain of revisions. Users may wish to go back and fork from an intermediate chart, preserve one promising path while testing another, or synthesize insights from multiple views into a new comparative visualization. Prior (non-visualization) provenance-inspired and generative-authoring systems such as Spellburst and CoQuest have shown the value of branching and combining intermediate outputs for creative exploration in tasks such as programming and brainstorming [6, 29]. *Likewise, visualization-focused design tools should treat visualization states as reusable objects that users can branch from, duplicate, and merge, so that divergent analytical directions can be explored and later brought back into relation with one another.*

4 SYSTEM DESIGN AND IMPLEMENTATION

VISCANVAS is a visualization authoring tool that supports non-linear exploration via human-LLM collaboration, with an emphasis on supporting rapid visualization authoring that is common in the early stages of data analysis. Below, we describe the interface features and system implementation details of VISCANVAS (including how various features are designed to support C1-C3); evaluations are presented in Sec. 5. To support reproducibility, the system's codebase is available at <https://github.com/svl-at-asu/VisCanvas>.

4.1 User Interface & Features

The VISCANVAS interface (see Fig. 1 for the entire UI) is presented as a set of components on a zoomable and pannable canvas (like Miro or Figma). Users can create and manipulate different types of nodes, which are connected by edges to indicate the analysis workflow. Specifically, VISCANVAS supports four types of nodes (Text Input Node, Visualization Node, Merge Node, Menu Node). VISCANVAS also supports edge-specific operations and additional usability features such as semantic level-of-detail zooming and a data explorer panel for reviewing the dataset. Below, we describe these components and features and note how they support the design considerations introduced in Sec. 3.

Text Input Nodes: Through the Context Menu (Fig. 1-a), opened by right-clicking, users can add a Text Input Node to start a new graph. In this node, they can input natural language instructions to generate Vega-Lite specifications (Fig. 1-b) or modify an existing Vega-Lite specification (Fig. 3-a). The text input field supports autocompletion of the column names in the dataset (Fig. 1-b1), and users can also select the desired LLM for generating the Vega-Lite specifications (Fig. 1-b2). For testing, gpt-4.1-nano¹ and gpt-5-nano² were used, though the system is easily extensible to accommodate additional models. When a user submits a prompt from a Text Input Node, the system creates a new Visualization Node and connects it to the Text Input Node via an edge if one does not already exist. Upon receiving the backend response, the system updates the *Visualization Node's* specification; this update then cascades through all connected downstream nodes, enabling the automatic revision of an entire visualization chain.

Visualization Nodes: Vega-Lite charts are rendered in Visualization Nodes (Fig. 1-c). Each node renders a standard Vega-Lite specification using Vega-Lite's renderer, enabling it to display interactive or composite visualizations, including basic interactions such as filtering and panning and zooming, as well as multiple linked views within a

¹<https://platform.openai.com/docs/models/gpt-4.1-nano>

²<https://platform.openai.com/docs/models/gpt-5-nano>

single node. In addition, Visualization Nodes offer several functionalities to help users understand, refine, and interact with the generated visualizations. For example, hovering over an icon reveals the rationale for an LLM-generated visualization (Fig. 2-a), clarifying the system’s design choices [20]. Users can further refine the visualization through two editing interfaces: a *Visual Builder* (Fig. 2-b), a widget-based GUI for modifying the specification without writing code, and a *Vega-Lite Editor* (Fig. 2-c) for directly editing the specification as raw text. These GUI editors are intended to complement natural language instructions by offering fine-grained control, immediate visual feedback, and quick undo/redo of edits [50]. Users can also bookmark a visualization and leave a note for later reference (Fig. 2-d); bookmarked visualizations are displayed in a Bookmark Panel (Fig. 1-d).

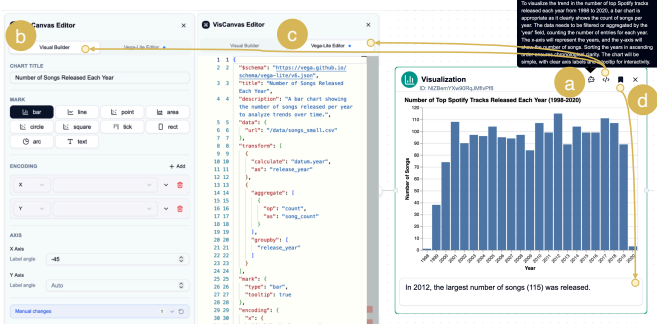


Figure 2: Visualization Node: (a) Robot icon shows the rationale for the LLM-generated results; (b) Visual Builder and (c) Vega-Lite Editor are provided for editing the Vega-Lite specification; (d) Bookmark icon allows users to bookmark the visualization for later reference.

Menu Nodes [C2, C3]: Menu Nodes provide four operations for refining and extending the exploration (Fig. 1-e). *Modify* adds a new Text Input Node as a child of a parent Visualization Node, allowing users to issue new instructions that further explore or refine the existing visualization (Fig. 3-a). *Duplicate* creates a copy of the parent Visualization Node as a new independent node in the graph, allowing users to edit the visualization while keeping the original visualization (Fig. 3-b). *Merge* adds a Merge Node and allows users to combine two Visualization Nodes (labeled Source 1 and 2) into a single comparative visualization (Fig. 3-c): Source 1 is automatically set to the parent node, while Source 2 can be specified by entering a node ID or by activating cursor mode and clicking the target node (C3). The *Merge* operation is prompt-guided rather than based on a hand-coded lookup table over source chart types. The prompt asks the LLM to identify shared attributes, align shared axes when present, and generate a layered or dual-axis chart to support direct comparison, with layered charts limited to at most two layers for readability; otherwise, it constructs a related view, such as a repeated chart, that reveals relationships or contrasts between the two sources (see Appx. A.2). *Suggest* prompts the system to generate 3–5 candidate visualization instructions as next analysis steps (Fig. 3-d), grounded in the analysis goals (a by-product of Vega-Lite specification generation discussed in Sec. 4.2.2) accumulated along the path from the root to the current node; these suggestions are instantiated as persistent Text Input Nodes on the canvas, so users can return to them and adopt them later (C2).

Edge Dropdown Menus [C3]: Edge Dropdown Menus provide two operations for extending the exploration (Fig. 1-f). *Fill Blank* generates an intermediate Visualization Node that bridges the analytical gap between two connected Visualization Nodes (Fig. 4-a). Like the *Merge* operation, this operation is prompt-guided: the prompt asks the LLM to identify shared or related attributes, differences in analytical focus, and potential causal or correlational relationships between the two endpoint visualizations, then generate a Vega-Lite specification that makes the reasoning between them more explicit (see Appx. A.2). The generated node is inserted between the two existing nodes as a new Visualization Node. *Branch* handles on the target node of the selected edge: if the target is a Visualization Node, it creates a new independent

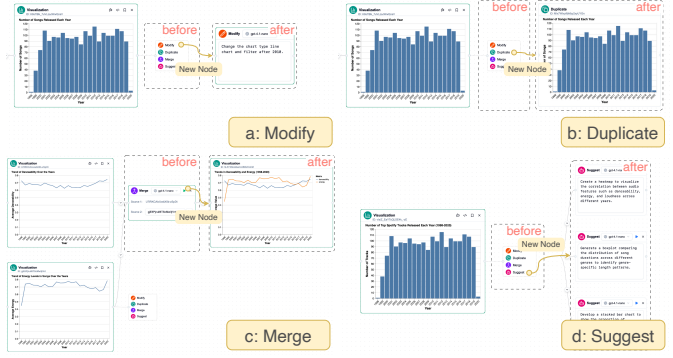


Figure 3: (a) *Modify* enables users to generate visualizations via natural language; (b) *Duplicate* clones the parent visualization; (c) *Merge* combines two distinct data views; (d) *Suggest* recommends 3–5 LLM-generated prompts to users.



Figure 4: (a) *Fill Blank* inserts an intermediate visualization between two connected nodes; (b) *Branch* creates a Menu Node or Visualization Node based on the edge’s target node.

copy; otherwise, it inserts a new Menu Node (Fig. 4-b). In either case, users can explore an alternative analysis direction without modifying the original graph structure.

Semantic Zoom [C1]: To support navigation as the exploration graph grows in size, VISCANVAS employs a semantic zoom feature, which adapts the level of detail shown for each node based on the current zoom level (Fig. 5). When zoomed out, all nodes are rendered as compact icons, providing a high-level overview of the graph structure (*Icon* zoom level). Zooming in changes nodes to the *Title/Field* zoom level, where Visualization Nodes display their title alongside the field names assigned to each encoding, allowing users to quickly identify the content of each chart without rendering the full visualization. At the *Full* zoom level, Visualization Nodes render the complete Vega-Lite chart. Text Input Nodes, Merge Nodes, and Menu Nodes maintain the same appearance at both the *Title/Field* and *Full* levels.

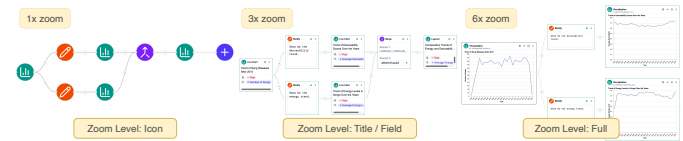


Figure 5: Semantic Zoom: Users can navigate large workflows by adjusting detail by zoom level.

Data Explorer: The Data Explorer panel is accessible via the *Show Data Explorer* button at the bottom of the canvas screen (Fig. 1-g), and comprises a *Data Preview* and *Column Details* pair of tabs. *Data Preview* (Fig. 6-a) displays the dataset in a tabular format, while the *Column Details* (Fig. 6-b) shows metadata for each column, including its description, data type, and data distribution.

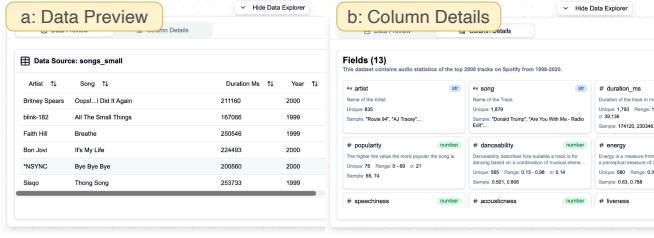


Figure 6: Data Explorer: Users can explore the datasets by (a) data preview and (b) column details.

4.2 Implementation Details

4.2.1 Frontend

The VISCANVAS frontend is implemented using a combination of React, Reactflow³, Zustand⁴, and elkjs⁵. React is the foundation for our user interfaces, while Zustand is used for state management of nodes and edges of the graph. For the graph visualization, we use Reactflow to build the node-based interface for exploratory visualization authoring. We also use elkjs to automatically arrange the graph in a hierarchical manner to improve the readability of the graph. Within Reactflow, rendering Vega-Lite visualizations as SVG caused repeated re-rendering during node movement and led to sluggish interactions, so we instead used canvas-based rendering to improve responsiveness. To make the editing experience more intuitive, the system updates visualizations in real-time as users modify specifications via the Visual Builder or Vega-Lite Editor. The automatic layout is triggered whenever new nodes are added or the graph dimensions change. An example of the JSON objects used to manage node and edge states is provided in Appx. C.

4.2.2 Backend

The VISCANVAS backend is implemented using FastAPI⁶ to orchestrate core system functionalities, including visualization specification generation and analysis goal suggestion. For the visualization specification generation and analysis goal suggestion, we utilize LangGraph⁷ to coordinate multiple LLM-based agents. An overview of the system flow is shown in Fig. 11-a,b in Appx. D; we detail the steps below.

Visualization Specification Generation: When a user submits either a natural language instruction via a Text Input Node or a predefined visualization instruction for the *Merge* and *Fill Blank* operations, the pipeline takes the instruction and any existing Vega-Lite specifications from upstream nodes as input. Because each operation is handled as a separate pipeline invocation, VISCANVAS does not maintain a persistent LLM session or implicit chat history across interactions. Instead, each operation issues a new LLM call with the relevant graph context explicitly supplied in the prompt. Accordingly, the current pipeline is designed for user instructions that can be interpreted as requests to produce or revise a Vega-Lite specification, rather than for open-ended multi-turn dialogue in which the system asks follow-up questions before generating a visualization. This pipeline comprises three LLM-powered modules orchestrated using LangGraph (Fig. 11-c): *Data Summarizer*, *Analysis Goal Generator*, and *Vega-Lite Spec Generator*.

Based on empirical review during development and testing, we employ gpt-4.1-nano for the *Data Summarizer* and *Analysis Goal Generator* modules, and either gpt-4.1-nano or gpt-5-nano for the *Vega-Lite Spec Generator* module; users can select the model for each Text Input Node as described in Sec. 4.1 (see also Sec. 5.2.5 for an analysis of these models' performance during our evaluations).

The *Data Summarizer* module analyzes the uploaded dataset and generates a compact, information-dense summary to serve as ground-

ing context for subsequent LLM modules. Although passing an entire dataset directly to the LLM is feasible with recent long-context models, doing so would significantly increase response latency and incur unnecessary cost, particularly as datasets grow in size and dimensionality. To this end, we adopt the Data Summarizer from LIDA [15], which extracts dataset properties including atomic types, general statistics, and representative values, enriched with LLM-generated semantic descriptions for the dataset and fields. Note that the Data Summarizer is invoked only once when a dataset is first uploaded; thereafter, the summary is cached and reused across all subsequent steps.

The *Analysis Goal Generator* module produces an intermediate analysis goal that bridges the user's natural language instruction and the final visualization specification. Given the data summary, the user's visualization instruction, and any existing Vega-Lite specifications from upstream nodes (if present), the module generates a concise analysis goal that captures the analytical intent of the request (see Appx. A.1 for the full prompt).

Finally, the *Vega-Lite Spec Generator* module transforms the analysis goal into a valid Vega-Lite specification. Rather than selecting or scoring chart-type-specific templates, the module uses a single generic Vega-Lite template with `<stub>` fields, which provides a flexible structure for generating different chart designs. Since LLMs have been shown to lack sufficient knowledge of Vega-Lite grammar [51], this module also provides curated Vega-Lite specification examples in the prompt as few-shot guidance toward well-formed output structures. It also outputs a short *rationale* explaining the visualization design to help users understand the intent behind the LLM's output. To improve reliability, the module incorporates a self-reflection mechanism: if the generated specification fails to compile, the module captures the error message and re-prompts the LLM to diagnose and correct the issue, iterating until a valid specification is produced (see Appx. A.2 for the full prompt).

The frontend displays the generated Vega-Lite specification and rationale to the user in a Visualization Node. In addition, the frontend stores the analysis goal in each Visualization Node for the analysis goal suggestion.

Analysis Goal Suggestion: When a user selects the *Suggest* option on a Menu Node, the system collects the analysis goals stored in the Visualization Nodes along the path from the root to the current node. These accumulated goals, together with the cached data summary, are provided to gpt-4.1-nano, which generates 3–5 candidate visualization instructions as relevant next analysis steps based on the context of the preceding exploration path (see Appx. B for the full prompt). The suggested instructions are then presented as Text Input Nodes, which users can directly adopt to continue their exploration.

5 EVALUATION

5.1 User Study Design

To empirically evaluate VISCANVAS, we conducted a within-subjects study with 20 participants. During each session, participants performed two sets of exploratory analysis tasks requiring them to generate Vega-Lite visualizations and extract distinct insights from the data. For each participant, one task was conducted using VISCANVAS, and the other using a baseline, chat-based interface.

Experimental Baseline. We developed VISCHAT as a controlled chat-based baseline for testing against VISCANVAS. Rather than a literal replica of commercial systems such as ChatGPT or Gemini, VISCHAT preserves their linear chat layout while matching key capabilities available in VISCANVAS. Specifically, VISCHAT includes the same *Suggest* backend, Visual Builder, and Vega-Lite Editor as VISCANVAS, controlling for access to LLM-generated next-step recommendations and low-level chart refinement. The two interfaces expose these capabilities through interface-native mechanisms: suggestions appear as editable prompt candidates in VISCHAT, whereas they become Text Input Nodes in VISCANVAS. Thus, the primary comparison focuses on the linear chat interface versus the node-based spatial interface.

Research Questions. To help scaffold our study's analyses and interpretations, we posed the following research questions:

³ <https://reactflow.dev/>

⁴ <https://zustand.docs.pmnd.rs/>

⁵ <https://github.com/kieler/elkjs>

⁶ <https://fastapi.tiangolo.com/>

⁷ <https://www.langchain.com/langgraph>

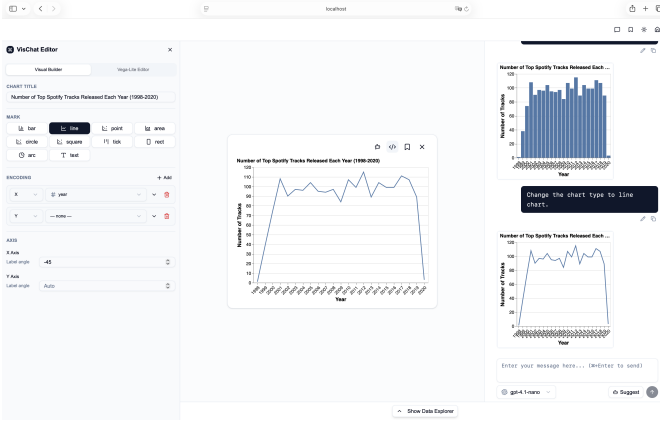


Figure 7: VISCHAT, a controlled chat-based authoring baseline that preserves the familiar linear layout of commercial LLM chat interfaces while including suggestion and visualization-editing capabilities. The right panel shows the chart and chat history; the middle and left panels show the Visual Builder for editing the current chart design and its properties.

- **RQ1:** Does a node-based interface (i.e., VISCANVAS) make exploratory visual analysis easier or harder than a linear chat-based interface (i.e., VISCHAT)?
- **RQ2:** Do users prefer a node-based interface over a linear chat-based interface for exploratory visual analysis?
- **RQ3:** What features or interactions in VISCANVAS are considered beneficial or detrimental to exploratory visual analysis?
- **RQ4:** What strategies do users employ during exploratory visual analysis with a node-based vs. a linear chat-based interface?
- **RQ5:** Is VISCANVAS’s backend architecture and system flow sufficiently responsive and performant for interactive use?

Datasets. To evaluate VISCANVAS across varied contexts, we chose three datasets based on their size, complexity, and high potential for participant engagement: an *Automobile* dataset⁸ (used exclusively for warm-up exercises), a *Spotify* dataset⁹, and a *Social Media & Mental Health* dataset¹⁰. The latter two were used for the main tasks, with their order and the assigned interface (VISCANVAS vs. VISCHAT) counterbalanced to mitigate order or data preference effects. The *Spotify* dataset has 2,000 records and 13 variables (3 nominal, 1 temporal, 9 quantitative). The *Social Media & Mental Health* dataset has 705 records and 13 variables (6 nominal, 7 quantitative). One participant reported that they had seen the *Spotify* dataset before the study (their activities on this dataset were not unusual compared to other participants, so we included their results). All datasets are available in our supplemental [OSF repository](#).

Study Procedure. The study was conducted in real time via Zoom to allow participants to work in their preferred environment. Sessions were expected to last 90 minutes; they averaged 86.8 minutes and ranged from 66 to 107 minutes. After an introductory phase, participants completed two conditions (in random order). For each condition, they watched a tutorial video, completed a warm-up exercise, performed the main task, and filled out a post-task survey. An optional break was provided between the two conditions, and a final post-study survey was administered at the end. This study was classified as non-human-subjects research by our university’s Institutional Review Board (IRB); nevertheless, we obtained informed consent from all participants.

Introduction and Consent: Participants provided informed consent prior to the session. At the start of the session, the facilitator introduced the study’s purpose and provided an overview of the interface features (1–2 minutes), followed by a pre-study demographic survey (5 minutes).

⁸<https://www.kaggle.com/datasets/athirags/car-data>

⁹<https://www.kaggle.com/datasets/paradisejoy/top-hits-spotify-from-20002019>

¹⁰<https://www.kaggle.com/code/nuhmanpk/student-social-media-and-relationship-analysis>

Tutorial Video and Warm-up (per condition): To minimize facilitator effects, participants viewed a prerecorded demonstration video (approx. 2 minutes) for the assigned condition, followed by a hands-on warm-up period (5–7 minutes) to explore the features and interaction patterns of the interface.

Main Task (per condition): Each participant conducted a free-form exploratory analysis of the assigned dataset (limited to 20 minutes), aiming to produce visualizations and record at least four bookmarked insights. To promote engagement with the analysis, participants were asked to give a brief oral report of their findings at the end of each condition, referencing the visualizations they had bookmarked. We did not divide the exploratory analysis task into subtasks (i.e., exploration & question answering), preserving the natural exploratory process [7].

Post-Task Survey (per condition): After completing the main task, participants filled out a short questionnaire that included the NASA-TLX to assess workload with additional questions to evaluate usability.

Break: An optional break (3–5 minutes) was provided between the two conditions to mitigate fatigue. Participants then proceeded to the second condition using the alternate interface and dataset (i.e., if they first used VISCANVAS with the Spotify dataset, they next used VisChat with the Social Media dataset).

Post-Study Survey and Debrief: After completing both conditions, participants filled out a final post-study survey (5–10 minutes) covering overall interface preferences and open-ended feedback. Immediately after the survey, we asked them whether they had any additional feedback or questions (1–2 minutes).

Participants. We recruited 20 graduate computer science students (13 males, 7 females; age: $\mu = 23.25, \sigma = 1.52$) from Arizona State University (P1–P20). While all participants were familiar with data visualization tools like D3.js, their experience with other technologies varied (see Appx. E for full details). Most were frequent users of LLMs (90% at least weekly) and possessed high self-reported proficiency in data visualization (85%) and data analysis (75%). However, this expertise did not fully extend to the intersection of these fields; only 40% used LLMs specifically for visualization tasks on a weekly basis. Furthermore, despite VISCANVAS utilizing a canvas-based interaction paradigm similar to Miro or Figma, participants reported relatively little experience with such whiteboard-style formats (16% at least weekly).

5.2 Analysis & Results

During the study, we collected both quantitative and qualitative information, including task- and usage-related information, survey responses, open-ended freeform feedback, and system logging of participant actions. We report and discuss these results in the following sections (with the p -values from the Wilcoxon signed-rank tests provided in parentheses where applicable).

5.2.1 Assessing Task Success and Perceived Difficulty (RQ1)

As an initial sanity check, we first verified that participants could complete the *Main Task* stages using the interfaces. For both interfaces, all participants successfully created at least four visualizations, which was considered “success” in our study design.

In terms of how “easy” participants considered it was to complete each task stage, in the *Post-Task Surveys* participants reported comparable levels of task completion ease for both VISCANVAS ($\mu = 6, \sigma = 1.34$) and VISCHAT ($\mu = 6.05, \sigma = 1.12$), with no significant difference between the two conditions (p -value = 0.89). Similarly, participants expressed comparable confidence in the thoroughness of their analysis in VISCANVAS ($\mu = 5.8, \sigma = 0.93$) and VISCHAT ($\mu = 5.75, \sigma = 1.04$), again with no significant difference (p -value = 0.80) (see Fig. 8). These results are not surprising, given that participants performed an open-ended task that was capped at 20 minutes, with a relatively small number of required visualization bookmarks. To better understand how each interface promoted different exploration strategies, we subsequently conducted a more detailed analysis of user authoring behavior (see Sec. 5.2.4).

5.2.2 Workload and Feature Assessments (RQ1)

In each *Post-Task Survey*, participants also completed NASA-TLX ratings and a set of system feature assessments for each condition (VISCANVAS vs. VISCHAT). For TLX scores (see Fig. 8), pairwise Wilcoxon signed-rank tests revealed no statistically significant differences on any of the measured dimensions: mental demand, temporal demand, performance, effort, and frustration, which suggests that subjective workload was comparable across the two conditions, despite the fact that, as reported in Sec. 5.1, most participants had limited prior experience with canvas-style tools, and VISCANVAS introduced a richer spatial interaction model than the sequential chat layout of VISCHAT.

For the system feature assessments (see Fig. 9), VISCANVAS received more favorable ratings than VISCHAT across nearly all assessed features. However, pairwise Wilcoxon signed-rank tests did not yield widespread statistical significance with Type I error controlled at the 0.05 level, with the notable exception that VISCANVAS received significantly higher ratings for supporting the exploration of multiple analysis directions in parallel. One possible explanation for the limited statistical differences is participants' familiarity with linear, chat-based systems; whereas VISCHAT followed a familiar sequential interaction model, VISCANVAS introduced a richer and less familiar spatial interaction model. Nevertheless, participants generally rated VISCANVAS's exploration-focused features favorably, without reporting significantly greater workload or frustration. These assessments are further contextualized by participants' freeform feedback in Sec. 5.2.3.

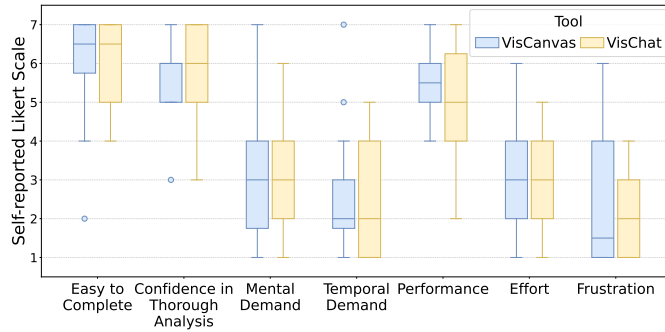


Figure 8: Participants' self-reported scores for NASA TLX questions, ease of completing the task, and confidence in their analysis thoroughness.

5.2.3 User Preferences and Freeform Feedback (RQ2, RQ3)

In the *Post-Study Survey and Debrief* stage, participants were first asked about their preferences for VISCANVAS and VISCHAT for (i) open-ended exploratory analysis tasks and for (ii) closed, targeted analysis tasks (such as constructing a targeted chart to directly answer a hypothesis), using a 7-point Likert scale where 1 signifies a preference for VISCHAT and 7 indicates a preference for VISCANVAS.

For open-ended analysis, responses were highly skewed toward VISCANVAS ($\mu = 5.35, \sigma = 2.22$): 14/20 participants rated above the midpoint, and 11/20 participants selected the strongest preference for VISCANVAS (a score of 7). In contrast, preferences for the closed and targeted analysis tasks were more mixed ($\mu = 3.8, \sigma = 2.16$): 8/20 participants preferred VISCANVAS, 10/20 preferred VISCHAT, and 2/20 selected the midpoint (indicating no clear overall preference).

Freeform feedback shed further light on how participants preferred to use and benefit from the tools, such as the features and interactions considered beneficial or detrimental to success. To analyze this feedback, we conducted a thematic analysis, which is summarized into the following high-level findings.

Finding 1 [C1]: Spatial visibility and semantic zoom supported navigation and thought organization. Several participants emphasized that the canvas-based interface helped them navigate their analysis and keep track of the overall analysis flow. P3 noted, “*VisCanvas is much more open than VisChat, and I can have a better view of where*

I have been and where I am going. In VisChat I had to scroll through the chat history to see what I had done.” Similarly, P11 commented on VISCANVAS, “*I can clearly see what I prompted and what the LLM generated.*” In addition, P4 and P6 noted that the canvas-based interface helped them structure their thoughts in a “*tree format*” while keeping past visualizations visible and organized for quick reference and easy access. P9 also highlighted semantic zoom as useful “*to focus on certain visualizations.*” However, two participants suggested making the zoom level lockable and manually toggleable as a way for more fine-grained control.

Finding 2 [C2, C3]: Canvas-based interface encouraged parallel exploration. VISCANVAS's canvas-based interface promoted tracking and pursuing multiple analytical directions in parallel. P9 explained, “*VisCanvas helped map out my ideas and go into several different directions at once,*” and P2 discussed how, “*with VisCanvas, we can have all possible suggestions and their visualizations on a single screen.*” This result is further supported by the post-task survey (see Fig. 9): participants indicated that VISCANVAS significantly enhanced their ability to pursue multiple analysis directions simultaneously compared to VISCHAT ($p = 0.01$).

Finding 3 [C3]: Comparison and merging encouraged new insights. Several participants mentioned VISCANVAS made it easier to merge and compare visualizations, which in turn encouraged them to ideate new directions and ideas. For example, P18 noted, “*having well-defined graphs sit next to each other for different questions was very useful for further analysis.*” Relatedly, P16 described *Fill Blank* as useful because it “*filled [the] gap between two charts,*” helping users understand their connectivity and view related charts together. This trend is in part supported by the post-task survey (see Fig. 9), where participants indicated that VISCANVAS helped them somewhat more than VISCHAT to obtain new insights by merging and comparing visualizations, though this difference did not reach statistical significance ($p = 0.07$).

Finding 4: Familiarity and lower interaction overhead made chat-based interfaces appealing. Some participants preferred VISCHAT even for open-ended exploratory tasks, citing its lower interaction overhead, more familiar interaction style, and lower learning curve. In the context of open-ended analysis, P16 explained, “*VisChat would allow me to do that easily because of the familiar chat interface,*” and P7 similarly mentioned, “*I would prefer VisChat because it was easier to use.*” While our *Post-Study Survey* preference results were 14/20 in favor of VISCANVAS for open-ended tasks, these comments highlight that, for at least some participants, there was a tension between the more complex user experience and feature set of VISCANVAS and the sequential, chat-based familiarity of VISCHAT.

Finding 5: Linear focus and straightforward interaction supported targeted analysis. When reflecting on its potential for targeted analysis, participants more often described VISCHAT as better suited for pursuing predefined questions or validating hypotheses. Several participants mentioned that its straightforward and intuitive interaction style matched a focused analysis flow. For instance, P9 noted, “*VisChat's interface is set up better to explore a single chain of thought compared to VisCanvas,*” and P19 commented, “*I would prefer VisChat for targeted analysis since we can focus on one single chart at a time and, with a predefined question, I can focus on that.*”

Finding 6 [C1]: Visibility and control over intermediate results remained valuable in targeted analysis. At the same time, a subset of participants preferred VISCANVAS in targeted exploration when they valued visibility and control across intermediate results. P17 explained, “*I can see most of my plots on screen at the same time rather than scrolling up a chat,*” and P12 appreciated its incremental workflow, noting that the step-by-step creation of graphs could be a “*game changer*” in more complex analytical scenarios.

5.2.4 User Strategies for Exploratory Visual Authoring (RQ4)

System logs showed no significant between-interface differences in Visualization Nodes created during the *Main Task* (VISCANVAS: $\mu = 8.95, \sigma = 3.56$; VISCHAT: $\mu = 7.95, \sigma = 4.26$) or unique variable sets explored (VISCANVAS: $\mu = 7.9, \sigma = 3.71$; VISCHAT: $\mu = 8.4,$

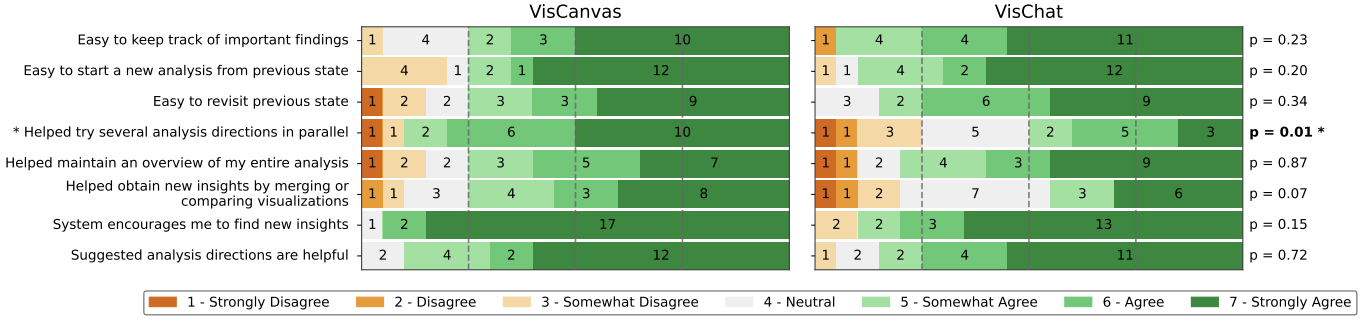


Figure 9: System Evaluation Results. Likert scale responses from the post-study survey. Wilcoxon signed-rank p -values are provided to the right.

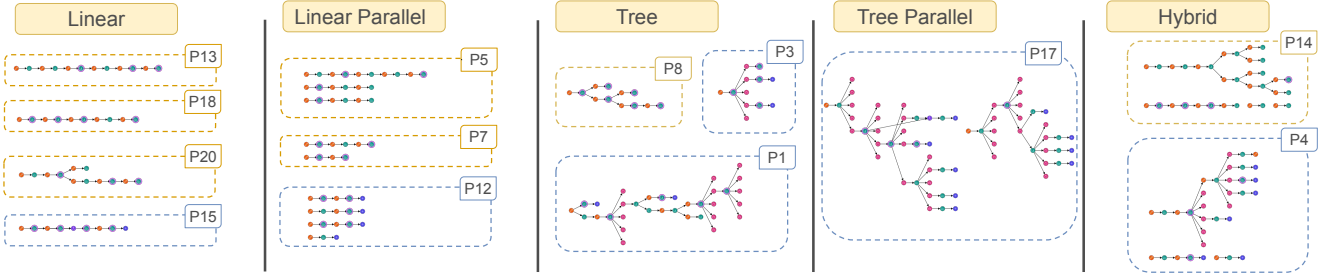


Figure 10: Representative examples of user exploration strategies, ordered by increasing structural complexity (left to right). Node colors correspond to their type: ● Text, ● Visualization, ● Merge, ● Suggest, and ● Menu nodes. VISCHAT (yellow border) predominantly drove constrained, Linear explorations, whereas VISCANVAS (blue border) supported more divergent, complex behaviors, dominating the Hybrid category and generating the sole Tree Parallel structure.

$\sigma = 5.09$). However, to better characterize the exploration and chart creation strategies of participants, we analyzed the topologies of their exploration structures (see Fig. 10). Based on a collaborative review by two researchers, we derived the following taxonomy of exploration patterns that were commonly performed by our participants:

- **Linear:** A structure that is strictly or semantically linear.
- **Linear Parallel:** Two or more concurrent linear explorations.
- **Tree:** A singular hierarchical structure (i.e., one parent node with two or more child nodes).
- **Tree Parallel:** Two or more concurrent tree structures.
- **Hybrid:** Any combination of tree and linear structures.

To classify these structures, one author developed a codebook from the taxonomy, and three authors independently coded all participant topologies, achieving 82.5% initial agreement; disagreements were resolved through discussion. The complete set of participant topologies is provided in Appx. F alongside visualizations of the distributions.

Data classification revealed distinct distributional trends when contrasting the two interfaces. VISCANVAS showed a clear preference in *Hybrid* exploration (9 vs. 4 for VISCHAT) and *Tree*-based exploration (5 vs. 2). Conversely, *Linear* exploration was heavily favored in VISCHAT (8 vs. 1). These results align with our initial notions regarding the navigational affordances of each interface, and support our goal of exploratory analyses via VISCANVAS.

Only one interface, VISCANVAS, was used for the *Tree Parallel* topology (only created once). For *Linear Parallel* exploration, the counts were similar (4 for VISCANVAS vs. 6 for VISCHAT), which we consider comparable given the sample size. Finally, exploration approaches were consistent across datasets, with at most a one-participant difference per category.

A closer inspection of target node types at branching points further clarifies how VISCANVAS’s features contributed to these exploration structures. In this analysis, we counted *Suggest* nodes only when the suggested prompt was subsequently connected to a Visualization Node, thereby excluding suggestions that were generated but not pursued. Among branch targets that led to visualization results, Visualization

Nodes accounted for the largest share (42/91, 46.2%), followed closely by adopted *Suggest* nodes (37/91, 40.7%). *Modify* nodes accounted for a smaller share (7/91, 7.7%), while *Fill Blank* (2/91, 2.2%), *Merge* (2/91, 2.2%), and *Duplicate* (1/91, 1.1%) were comparatively rare as branch targets. These results suggest that *Suggest* played a substantial role in supporting branching exploration, while branching also emerged through the *Branch* operation in Edge Dropdown Menus and the *Modify* operation in Menu Nodes.

5.2.5 LLM Model Usage & Performance (RQ5)

Finally, to evaluate the system performance of VISCANVAS, we analyzed the response times of all visualization generation requests made during the study sessions. Across 20 participants, VISCANVAS received 291 visualization generation requests, each of which triggered the visualization generation pipeline described in Sec. 4.2.2. gpt-4.1-nano handled the majority of requests ($n = 261$, 89.7%), while gpt-5-nano was used for the remaining requests ($n = 30$, 10.3%), selected by 10/20 participants for a subset of their interactions. The mean end-to-end response time was 6.70s ($\sigma = 4.08$ s) for gpt-4.1-nano and 32.78s ($\sigma = 15.61$ s) for gpt-5-nano. The latency of gpt-4.1-nano was sufficiently low to maintain interactive responsiveness, while the longer response times of gpt-5-nano reflect the greater reasoning demands of complex specification generation tasks. We define success rate as the proportion of generated Vega-Lite specifications that compiled without errors; gpt-4.1-nano achieved 96.2% and gpt-5-nano achieved 96.7%, indicating consistently reliable output across both models.

Participants’ open-ended feedback indicated that waiting times were generally acceptable. P7 noted, “I was primarily using the gpt-4.1-nano which generated the results fast.” Even P16, who used gpt-5-nano on several occasions, mentioned, “gpt-5-nano obviously took some more time, but overall the LLM timing was okay.”

While waiting for LLM responses, two distinct behavioral patterns were observed: passively waiting while focusing on the anticipated output, or actively exploring the data and system features in parallel. P10 noted, “I just thought about whether it would generate the correct visualization or not,” and P11 similarly mentioned, “I was just wondering if

the graph would look how I had pictured it.” In contrast, P6 mentioned, “I was thinking about which other charts could be generated while I was waiting for the LLM’s response,” and P18 noted, “While I was waiting, I simply did something else on the system since I was still exploring all the system’s features.” These observations indicate that participants did not uniformly wait idly, but rather that some actively leveraged wait time to pursue parallel analyses or explore system functionality.

6 DISCUSSION

VISCANVAS is an LLM-powered node-based exploratory visualization authoring tool that externalizes intermediate visualization states, branching paths, and relationships among analytical directions. Our evaluation results suggest that the majority of participants preferred VISCANVAS over the baseline chat-based interface, particularly for open-ended analysis tasks, and also that VISCANVAS supports the design considerations in Sec. 3 (C1–C3) by making exploration structures visible, supporting branching and comparison, and helping users move between broad exploration and focused refinement. More specifically, C1 is supported by Finding 1 and the topology analysis: participants described the canvas and semantic zoom as useful for seeing where they had been and where they could go next, and VISCANVAS produced more Tree and Hybrid exploration structures than VISCHAT. C2 is supported by Finding 2 and the topology analysis: participants reported that VISCANVAS helped them view possible suggestions and pursue several directions at once, and adopted *Suggest* prompts formed a substantial share of branch targets that led to visualizations. Because VISCHAT used the same suggestion backend, this difference cannot be attributed to higher suggestion quality in VISCANVAS. Instead, it suggests that VISCANVAS helped users preserve recommendations as persistent, spatially organized alternatives for parallel exploration. C3 is supported by Findings 2, 3 and the topology analysis, which together show that participants used VISCANVAS to pursue multiple directions in parallel and to compare or merge visualization states into new analysis directions. Below, we reflect on several lessons learned from our work and discuss opportunities for future work in this area.

Current System Limitations. The current system uses a data summary rather than the entire dataset to generate analysis goals and Vega-Lite specifications. This approach was chosen to prioritize response time, though it may limit the accuracy of the generated visualization specifications or analysis goal suggestions, particularly for large or complex datasets. Although no participant reached the model context-window limit or encountered context-length errors in our study, prompt size may increase for datasets with many columns, long root-to-node analysis paths used by the *Suggest* operation, or complex Vega-Lite specifications passed as graph context. Future work could address this limitation through explicit token-budget monitoring, context pruning, or context compaction for large or complex analyses. A related limitation is that VISCANVAS is not designed as a general-purpose multi-turn conversational agent. The current pipeline does not ask follow-up clarification questions before generating a visualization, nor does it support open-ended conversational exchanges such as discussing chart design alternatives without producing a Vega-Lite output. Future work could address this by introducing a new type of Text Input Node that supports multi-turn conversations with the agent.

User Study Limitations. Our user study also has several limitations. Participants were all graduate computer science students, and many reported relatively high familiarity with data analysis and visualization tools, limiting the generalizability of our findings to novices, domain experts, and broader populations. Moreover, the 20-minute sessions and requirement to bookmark at least four insights may have constrained natural analysis strategies and limited our ability to observe longer-term behaviors such as revisiting branches or using provenance over extended sensemaking. The low NASA-TLX ratings further suggest that the tasks were manageable for these high-skill participants, potentially introducing ceiling/floor effects in task success, perceived difficulty, and workload; future evaluations should involve more diverse participants and longer, more challenging analysis tasks.

Beyond these limitations, we identified several promising directions for future work.

Trade-offs of Node-based Exploration. Our results do not imply that node-based interfaces are universally preferable to linear chat workflows. While VISCANVAS did not significantly increase NASA-TLX workload, several participants preferred VISCHAT for its familiarity, lower interaction overhead, and focused single-chain workflow, particularly for targeted analysis tasks. These findings suggest that node-based provenance can introduce navigation and orientation costs as graphs grow, and future systems should consider a hybrid of both interfaces.

Enriching the Design Space. Our current system focuses on Vega-Lite, as it efficiently supports visualization generation and modification. However, this constrains the design space to chart types, data transformations, and interactions that fit within Vega-Lite’s abstraction. Future work could extend this approach to more expressive visualization grammars such as ECharts, Vega, or D3, which support richer custom interactions and more bespoke visual designs.

Design-Oriented Visualization Authoring. Our work demonstrates that VISCANVAS effectively supports a branching and exploratory style of chart authoring. In a downstream workflow, users could first use VISCANVAS to identify promising analytical branches and bookmark key insights, then reuse the generated charts and insights as starting points for further refinement in presentation-oriented design tools. However, design-oriented visualization authoring also involves exploration: users may alternate between broad exploration and focused refinement [10], or design multiple alternatives in parallel [12, 17]. Future systems could therefore build on our branching and comparison mechanisms to support such scenarios, including the construction and refinement of infographic designs, pictorial visualizations, or presentation-oriented visual storytelling, where the goal encompasses not only analytical correctness but also aesthetic quality and communicative effectiveness.

Integrating Interaction-Augmented Instructions. Another promising direction is to enrich prompting with interaction-augmented instructions [43]. While our current system relies primarily on text input, multimodal inputs such as speech, sketching, or pointing gestures could help users express ambiguous intentions and high-level design goals more naturally. Grounding such inputs in the provenance graph may improve the efficiency and interpretability of human-AI co-authoring.

Collaborative Authoring. Finally, our provenance-based representation opens opportunities for collaborative visual authoring. The graph can serve as a shared record of explored directions and analytical decisions, supporting asynchronous collaboration where collaborators resume work from meaningful intermediate states [36], as well as synchronous collaboration where multiple users can inspect, branch, and discuss visualization states in real time, drawing on interaction patterns from tools such as Miro or Figma.

7 CONCLUSION

In this work, we introduced VISCANVAS, a node-based interface for exploratory visualization authoring with LLMs. Motivated by the mismatch between exploratory visual analysis workflows and prevailing linear chat interfaces, VISCANVAS integrates visualization authoring with exploration provenance on a non-linear canvas, where Vega-Lite visualization states can be created, revised, branched, compared, and merged. VISCANVAS combines a graph-centric interface with an LLM pipeline for generating visualizations and context-aware suggestions, helping users maintain multiple analytical directions while moving between focused refinement and breadth-oriented exploration. Our controlled study with 20 participants demonstrates that VISCANVAS promotes more diverse exploratory authoring behaviors and is preferred for open-ended analysis, while achieving levels of cognitive load and usability comparable with those of a chat-based baseline. Collectively, our work offers an initial step toward LLM-assisted visualization authoring environments that support complex, non-linear exploratory analysis workflows.

REFERENCES

- [1] Business Intelligence and Analytics Software. <https://www.tableau.com>. 1, 2
- [2] Colab. <https://developers.google.com/colab>. 2
- [3] Microsoft Excel | Free Online Spreadsheets Software. <https://www.microsoft.com/en-us/microsoft-365/excel>. 2
- [4] Power BI - Data Visualization | Microsoft Power Platform. <https://www.microsoft.com/en-us/power-platform/products/power-bi>. 2
- [5] RAWGraphs. <https://www.rawgraphs.io/>. 2
- [6] T. Angert, M. Suzara, J. Han, C. Pondoc, and H. Subramonyam. Spellburst: A Node-based Interface for Exploratory Creative Coding with Natural Language Prompts. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, pp. 1–22. Association for Computing Machinery, New York, NY, USA, Oct. 2023. doi: 10.1145/3586183.3606719 3
- [7] L. Battle and J. Heer. Characterizing Exploratory Visual Analysis: A Literature Review and Evaluation of Analytic Provenance in Tableau. *Computer Graphics Forum*, 38(3):145–159, July 2019. doi: 10.1111/cgf.13678 6
- [8] M. Bostock, V. Ogievetsky, and J. Heer. D³ Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, Dec. 2011. doi: 10.1109/TVCG.2011.185 1, 2
- [9] M. Brehmer and T. Munzner. A Multi-Level Typology of Abstract Visualization Tasks. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2376–2385, Dec. 2013. doi: 10.1109/TVCG.2013.124 1
- [10] B. Buxton. *Sketching User Experiences: Getting the Design Right and the Right Design*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2007. doi: 10.1016/B978-0-12-374037-3.X5043-3 9
- [11] S. P. Callahan, J. Freire, E. Santos, C. E. Scheidegger, C. T. Silva, and H. T. Vo. VisTrails: visualization meets data management. In *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, pp. 745–747. ACM, Chicago IL USA, June 2006. doi: 10.1145/1142473.1142574 3
- [12] B. Camburn, B. Dunlap, T. Gurjar, C. Hamon, M. Green, D. Jensen et al. A systematic method for design prototyping. *Journal of Mechanical Design*, 137(8):081102, Aug. 2015. doi: 10.1115/1.4030331 9
- [13] J. Chen, J. Wu, J. Guo, V. Mohanty, X. Li, J. P. Ono et al. InterChat: Enhancing Generative Visual Analytics using Multimodal Interactions. *Computer Graphics Forum*, 44(3):e70112, June 2025. doi: 10.1111/cgf.70112 2
- [14] Z. Chen, J. Chen, S. Ö. Arik, M. Sra, T. Pfister, and J. Yoon. CoDA: Agent Systems for Collaborative Data Visualization, Oct. 2025. doi: 10.48550/arXiv.2510.03194 2
- [15] V. Dibia. LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models. In D. Bollegala, R. Huang, and A. Ritter, eds., *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 113–126. Association for Computational Linguistics, Toronto, Canada, July 2023. doi: 10.18653/v1/2023.acl-demo.11 1, 2, 5
- [16] J. Doboš, N. J. Mitra, and A. Steed. 3D Timeline: Reverse engineering of a part-based provenance from consecutive 3d models. *Computer Graphics Forum*, 33(2):135–144, May 2014. doi: 10.1111/cgf.12311 3
- [17] S. P. Dow, A. Glassco, J. Kass, M. Schwarz, D. L. Schwartz, and S. R. Klemmer. Parallel prototyping leads to better design results, more divergence, and increased self-efficacy. *ACM Trans. Comput.-Hum. Interact.*, 17(4):18:1–18:24, Dec. 2011. doi: 10.1145/1879831.1879836 1, 9
- [18] C. Dunne, N. Henry Riche, B. Lee, R. Metoyer, and G. Robertson. GraphTrail: analyzing large multivariate, heterogeneous networks while supporting exploration history. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '12, pp. 1663–1672. Association for Computing Machinery, New York, NY, USA, May 2012. doi: 10.1145/2207676.2208293 3
- [19] K. Goswami, P. Mathur, R. Rossi, and F. Derroncourt. PlotGen: Multi-Agent LLM-based Scientific Data Visualization via Multimodal Retrieval Feedback. In *Companion Proceedings of the ACM on Web Conference 2025*, WWW '25, pp. 1672–1676. Association for Computing Machinery, New York, NY, USA, May 2025. doi: 10.1145/3701716.3716888 2
- [20] K. Gu, R. Shang, T. Althoff, C. Wang, and S. M. Drucker. How Do Analysts Understand and Verify AI-Assisted Data Analyses? In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, pp. 1–22. Association for Computing Machinery, New York, NY, USA, May 2024. doi: 10.1145/3613904.3642497 3, 4
- [21] Y. Guo, H. Shao, C. Liu, K. Xu, and X. Yuan. PromptTHIS: Visualizing the Process and Influence of Prompt Editing During Text-to-Image Creation. *IEEE Transactions on Visualization and Computer Graphics*, 31(9):4547–4559, Sept. 2025. doi: 10.1109/TVCG.2024.3408255 3
- [22] Y. Han, C. Zhang, X. Chen, X. Yang, Z. Wang, G. Yu et al. ChartLlama: A Multimodal LLM for Chart Understanding and Generation, Nov. 2023. doi: 10.48550/arXiv.2311.16483 1, 2
- [23] J. Heer, J. Mackinlay, C. Stolte, and M. Agrawala. Graphical Histories for Visualization: Supporting Analysis, Communication, and Evaluation. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1189–1196, Nov. 2008. doi: 10.1109/TVCG.2008.137 2, 3
- [24] M.-H. Hong and A. Crisan. Data Has Entered the Chat: How Data Workers Conduct Exploratory Visual Analytic Conversations with GenAI Agents. *ACM Trans. Interact. Intell. Syst.*, 15(4):21:1–21:40, Dec. 2025. doi: 10.1145/3744750 2
- [25] M. Hutchinson, R. Jianu, A. Slingsby, and P. Madhyastha. Foundation model assisted visual analytics: Opportunities and Challenges. *Computers & Graphics*, 130:104246, Aug. 2025. doi: 10.1016/j.cag.2025.104246 2
- [26] W. Javed and N. Elmquist. ExPlates: Spatializing Interactive Analysis to Scaffold Visual Exploration. *Computer Graphics Forum*, 32(3pt4):441–450, June 2013. doi: 10.1111/cgf.12131 3
- [27] J. Kim, S. Suh, L. B. Chilton, and H. Xia. Metaphorian: Leveraging Large Language Models to Support Extended Metaphor Creation for Science Writing. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, DIS '23, pp. 115–135. Association for Computing Machinery, New York, NY, USA, July 2023. doi: 10.1145/3563657.3595996 3
- [28] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic et al. Jupyter Notebooks—a publishing format for reproducible computational workflows. In *IOS Press*, pp. 87–90, 2016. doi: 10.3233/978-1-61499-649-1-87 2
- [29] Y. Liu, S. Chen, H. Cheng, M. Yu, X. Ran, A. Mo et al. How AI Processing Delays Foster Creativity: Exploring Research Question Co-Creation with an LLM-based Agent. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, pp. 1–25. Association for Computing Machinery, New York, NY, USA, May 2024. doi: 10.1145/3613904.3642698 3
- [30] Z. Liu, J. Thompson, A. Wilson, M. Dontcheva, J. Delorey, S. Grigg et al. Data Illustrator: Augmenting Vector Design Tools with Lazy Data Binding for Expressive Visualization Authoring. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, pp. 1–13. Association for Computing Machinery, New York, NY, USA, Apr. 2018. doi: 10.1145/3173574.3173697 1
- [31] S. L'Yi, Q. Wang, F. Lekschas, and N. Gehlenborg. Gosling: A Grammar-based Toolkit for Scalable and Interactive Genomics Data Visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):140–150, Jan. 2022. doi: 10.1109/TVCG.2021.3114876 1, 2
- [32] K.-L. Ma. Image graphs—a novel approach to visual data exploration. In *Proceedings Visualization '99 (Cat. No. 99CB37067)*, pp. 81–88, Oct. 1999. doi: 10.1109/VISUAL.1999.809871 3
- [33] P. Maddigan and T. Susnjak. Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex and GPT-3 Large Language Models. *IEEE Access*, 11:45181–45193, May 2023. doi: 10.1109/ACCESS.2023.3274199 1, 2
- [34] G. Moreira, L. Ferreira, C. Veiga, M. Hosseini, and F. Miranda. Urbanite: A Dataflow-Based Framework for Human-AI Interactive Alignment in Urban Visual Analytics. *IEEE Transactions on Visualization and Computer Graphics*, 32(01):1065–1075, Jan. 2026. doi: 10.1109/TVCG.2025.3634644 2
- [35] G. G. Méndez, M. A. Nacenta, and S. Vandenheste. iVoLVER: Interactive Visual Language for Visualization Extraction and Reconstruction. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, CHI '16, pp. 4073–4085. Association for Computing Machinery, New York, NY, USA, May 2016. doi: 10.1145/2858036.2858435 1
- [36] D. Park, M. Suhail, M. Zheng, C. Dunne, E. Ragan, and N. Elmquist. StoryFacets: A design study on storytelling with visualizations for collaborative data analysis. *Information Visualization*, 21(1):3–16, Jan. 2022. doi: 10.1177/14738716211032653 3, 9
- [37] E. D. Ragan, A. Endert, J. Sanyal, and J. Chen. Characterizing Provenance in Visualization and Data Analysis: An Organizational Framework of Provenance Types and Purposes. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):31–40, Jan. 2016. doi: 10.1109/TVCG.2015.2467551 3

- [38] D. Ren, B. Lee, and M. Brehmer. Charticulator: Interactive Construction of Bespoke Chart Layouts. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):789–799, Jan. 2019. doi: [10.1109/TVCG.2018.2865158](https://doi.org/10.1109/TVCG.2018.2865158) 1
- [39] A. Satyanarayan and J. Heer. Lyra: An Interactive Visualization Design Environment. *Computer Graphics Forum*, 33(3):351–360, June 2014. doi: [10.1111/cgf.12391](https://doi.org/10.1111/cgf.12391) 1
- [40] A. Satyanarayan, B. Lee, D. Ren, J. Heer, J. Stasko, J. Thompson et al. Critical Reflections on Visualization Authoring Systems. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):461–471, Jan. 2020. doi: [10.1109/TVCG.2019.2934281](https://doi.org/10.1109/TVCG.2019.2934281) 1, 2
- [41] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-Lite: A Grammar of Interactive Graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, Jan. 2017. doi: [10.1109/TVCG.2016.2599030](https://doi.org/10.1109/TVCG.2016.2599030) 1, 2
- [42] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive Vega: A Streaming Dataflow Architecture for Declarative Interactive Visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):659–668, Jan. 2016. doi: [10.1109/TVCG.2015.2467091](https://doi.org/10.1109/TVCG.2015.2467091) 1, 2
- [43] L. Shen, H. Li, Y. Wang, X. Xie, and H. Qu. Prompting Generative AI with Interaction-Augmented Instructions. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, CHI EA '25, pp. 1–9. Association for Computing Machinery, New York, NY, USA, Apr. 2025. doi: [10.1145/3706599.3720080](https://doi.org/10.1145/3706599.3720080) 2, 9
- [44] L. Shen, E. Shen, Y. Luo, X. Yang, X. Hu, X. Zhang et al. Towards Natural Language Interfaces for Data Visualization: A Survey. *IEEE Transactions on Visualization and Computer Graphics*, 29(6):3121–3144, June 2023. doi: [10.1109/TVCG.2022.3148007](https://doi.org/10.1109/TVCG.2022.3148007) 2
- [45] B. Shneiderman. Creativity support tools: accelerating discovery and innovation. *Commun. ACM*, 50(12):20–32, Dec. 2007. doi: [10.1145/1323688.1323689](https://doi.org/10.1145/1323688.1323689) 3
- [46] C. Stolte, D. Tang, and P. Hanrahan. Polaris: a system for query, analysis, and visualization of multidimensional relational databases. *IEEE Transactions on Visualization and Computer Graphics*, 8(1):52–65, Jan. 2002. doi: [10.1109/2945.981851](https://doi.org/10.1109/2945.981851) 2
- [47] S. Suh, B. Min, S. Palani, and H. Xia. Sensecape: Enabling Multilevel Exploration and Sensemaking with Large Language Models. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, pp. 1–18. Association for Computing Machinery, New York, NY, USA, Oct. 2023. doi: [10.1145/3586183.3606756](https://doi.org/10.1145/3586183.3606756) 3
- [48] Y. Tian, W. Cui, D. Deng, X. Yi, Y. Yang, H. Zhang et al. ChartGPT: Leveraging LLMs to Generate Charts From Abstract Natural Language. *IEEE Transactions on Visualization and Computer Graphics*, 31(3):1731–1745, Mar. 2025. doi: [10.1109/TVCG.2024.3368621](https://doi.org/10.1109/TVCG.2024.3368621) 1, 2, 3
- [49] P. Ulbrich, M. Waldner, K. Furmanová, S. M. Marques, D. Bednář, B. Kozlíková et al. sMolBoxes: Dataflow Model for Molecular Dynamics Exploration. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):581–590, Jan. 2023. doi: [10.1109/TVCG.2022.3209411](https://doi.org/10.1109/TVCG.2022.3209411) 3
- [50] P. Vaithilingam, E. L. Glassman, J. P. Inala, and C. Wang. DynaVis: Dynamically Synthesized UI Widgets for Visualization Editing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, pp. 1–17. Association for Computing Machinery, New York, NY, USA, May 2024. doi: [10.1145/3613904.3642639](https://doi.org/10.1145/3613904.3642639) 2, 3, 4
- [51] X. Wang, C. Liang, S. Zheng, J. Liang, G. Li, Y. Zhang et al. Visualization Generation with Large Language Models: An Evaluation, Dec. 2025. doi: [10.48550/arXiv.2401.11255](https://doi.org/10.48550/arXiv.2401.11255) 2, 5
- [52] Z. J. Wang, D. Munechika, S. Lee, and D. H. Chau. SuperNOVA: Design Strategies and Opportunities for Interactive Visualization in Computational Notebooks. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, CHI EA '24, pp. 1–17. Association for Computing Machinery, New York, NY, USA, May 2024. doi: [10.1145/3613905.3650848](https://doi.org/10.1145/3613905.3650848) 2
- [53] Z. Wen, L. Weng, Y. Tang, R. Zhang, Y. Liu, B. Pan et al. Exploring Multimodal Prompt for Visualization Authoring with Large Language Models. *IEEE Transactions on Visualization and Computer Graphics*, 2026. In press. doi: [10.1109/TVCG.2026.3701510](https://doi.org/10.1109/TVCG.2026.3701510) 2
- [54] R. W. White and R. A. Roth. *Exploratory search: Beyond the query-response paradigm*. Number 3. Morgan & Claypool Publishers, 2009. doi: [10.1007/978-3-031-02260-9](https://doi.org/10.1007/978-3-031-02260-9) 3
- [55] H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. doi: [10.1007/978-3-319-24277-4](https://doi.org/10.1007/978-3-319-24277-4) 1, 2
- [56] L. Wilkinson. *The Grammar of Graphics*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012. doi: [10.1007/978-3-642-21551-3](https://doi.org/10.1007/978-3-642-21551-3) 13 2
- [57] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager: Exploratory Analysis via Faceted Browsing of Visualization Recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, Jan. 2016. doi: [10.1109/TVCG.2015.2467191](https://doi.org/10.1109/TVCG.2015.2467191) 1, 2, 3
- [58] K. Wongsuphasawat, Z. Qu, D. Moritz, R. Chang, F. Ouk, A. Anand et al. Voyager 2: Augmenting Visual Analysis with Partial View Specifications. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, CHI '17, pp. 2648–2659. Association for Computing Machinery, New York, NY, USA, May 2017. doi: [10.1145/3025453.3025768](https://doi.org/10.1145/3025453.3025768) 2
- [59] K. Xu, A. Ottley, C. Walchshofer, M. Streit, R. Chang, and J. Wenskovitch. Survey on the Analysis of User Interactions and Visualization Provenance. *Computer Graphics Forum*, 39(3):757–783, July 2020. doi: [10.1111/cgf.14035](https://doi.org/10.1111/cgf.14035) 3
- [60] B. Yu and C. T. Silva. VisFlow - Web-based Visualization Framework for Tabular Data with a Subset Flow Model. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):251–260, Jan. 2017. doi: [10.1109/TVCG.2016.2598497](https://doi.org/10.1109/TVCG.2016.2598497) 3
- [61] B. Yu and C. T. Silva. FlowSense: A Natural Language Interface for Visual Data Exploration within a Dataflow System. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1–11, Jan. 2020. doi: [10.1109/TVCG.2019.2934668](https://doi.org/10.1109/TVCG.2019.2934668) 3
- [62] Z. Zhou, W. Wang, M. Guo, Y. Wang, and D. Gotz. A Design Space for Surfacing Content Recommendations in Visual Analytic Platforms. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):84–94, Jan. 2023. doi: [10.1109/TVCG.2022.3209445](https://doi.org/10.1109/TVCG.2022.3209445) 3

A PROMPTS FOR VISUALIZATION GENERATION

In each prompt, certain variables are enclosed within curly brackets and colored blue for easy recognition.

A.1 Prompt for Analysis Goal Generator

The following variables are used in this prompt:

- `state['data_summary']`: Data summary generated by Data Summarizer.
- `state['vis_instruction']`: Visualization instruction from the user.
- `state['input_vega_specs']`: Existing Vega-Lite specifications from upstream nodes (if present).

```
You are an experienced data analyst skilled in generating analysis goals.

# Task
Generate an analysis goal given the Data Summary, Visualization Instruction, and the Existing Vega-Lite Specifications (if provided).

# Input
- Data Summary: {json.dumps(state['data_summary'], indent=2)}
- Visualization Instruction: {state['vis_instruction']}
{f"- Existing Vega-Lite Specifications: {json.dumps(state['input_vega_specs'], indent=2)}" if state.get('input_vega_specs') else ""}

# Output
Return ONLY a string that represents the analysis goal.
Do not include any explanation, markdown formatting, or code blocks. Just return the raw string.
```

A.2 Prompt for Vega-Lite Spec Generator

The following variables are used in this prompt:

- `vega_lite_template`: A generic Vega-Lite template with `<stub>` fields to be filled by the LLM.

```
{
  "$schema":
    "https://vega.github.io/schema/vega-lite/v6.json",
  "title": "<stub>",
  "description": "<stub>",
  "data": {
    "url": "$data_url"
  },
  "transform": "<stub>",
  "mark": "<stub>",
  "encoding": "<stub>"
}
```

- `vega_lite_examples`: Example Vega-Lite specifications for few-shot prompting.
- `state['data_summary']`: Data summary generated by Data Summarizer.
- `state['analysis_goal']`: Analysis goal generated by Analysis Goal Generator.
- `state['vis_instruction']`: Visualization instruction from the user. For *Merge* and *Fill Blank*, the system uses the following predefined instructions.

– *Merge*

Integrate the two visualizations into a single comparative view. Identify shared attributes (e.g., common axis fields) and align them accordingly. If a shared axis exists, generate a layered or dual-axis chart to support direct comparison; if using a layered chart, limit it to at most two layers to keep

the chart readable. Otherwise, construct a visualization that meaningfully relates the two sources (e.g., a repeated chart) to reveal potential relationships or contrasts.

– *Fill Blank*

Generate an intermediate visualization that complements and connects the two given visualizations. Identify shared or related attributes, differences in analytical focus, and potential causal or correlational relationships. Create a visualization that fills the analytical gap, making the reasoning between the first and second visualization more explicit and interpretable.

- `state['input_vega_specs']`: Existing Vega-Lite specifications from upstream nodes (if present).
- `state['output_vega_spec']`: Generated Vega-Lite specification.
- `state['vega_error_message']`: Compile Error message.

A.2.1 Vega-Lite Spec Generator

You are a helpful assistant highly skilled in writing PERFECT Vega-Lite specifications for visualizations.

```
# Task
Fill in the '<stub>' sections of the Vega-Lite Template with the correct values to generate a visualization given the Data Summary, the Analysis Goal, Visualization Instruction (if provided), and Existing Vega-Lite Specifications (if provided).
```

```
# Input
- Vega-Lite Template: {vega_lite_template}
- Data Summary: {json.dumps(state['data_summary'], indent=2)}
- Analysis Goal: {state['analysis_goal']}
{f"- Visualization Instruction: {state['vis_instruction']}" if state.get('vis_instruction') else ""}
{f"- Existing Vega-Lite Specifications: {json.dumps(state['input_vega_specs'], indent=2)}" if state.get('input_vega_specs') else ""}
- Vega-Lite Specification Examples: {json.dumps(vega_lite_examples, indent=2)}
```

```
# Output
Return ONLY a rationale and a valid JSON object that represents the complete Vega-Lite specification.
Do not include any explanation, markdown formatting, or code blocks. Just return the raw JSON and a rationale string.
The output should be in the following format:
```

```
{
  "rationale": "Brief explanation of design choices",
  "vega_spec": <complete vega-lite specification>
}
```

Strict Rules

- The specification you write MUST FOLLOW VISUALIZATION BEST PRACTICES i.e. meet the specified goal, apply the right transformation, use the right visualization type, use the right data encoding, and use the right aesthetics (e.g., ensure axis are legible).
- The transformations you apply MUST be correct and the fields you use MUST be correct.
- The visualization CODE MUST BE CORRECT and MUST NOT CONTAIN ANY SYNTAX OR LOGIC ERRORS (e.g., it must consider the field types and use them correctly). You can refer to the Vega-Lite Specification Examples to see how to write the specification.
- You MUST first generate a brief plan for how you would solve the task e.g. what transformations you would apply e.g. if you need to construct a new column, what fields you would use, what visualization type you would use, what aesthetics you would use, etc.

A.2.2 Reflection

```
You are an expert in fixing Vega-Lite specifications.

# Task
Fix the Vega-Lite specification to make it run successfully.

# Input
- Vega-Lite Specification: {state['output_vega_spec']}
- Error Message: {state['vega_error_message']}

# Output
Return ONLY a valid JSON object that represents the fixed
Vega-Lite specification.
Do not include any explanation, markdown formatting, or code
blocks. Just return the raw JSON.
The output should be in the following format:
{{
  "vega_spec": <fixed vega-lite specification>
}}
```

B PROMPT FOR ANALYSIS GOAL SUGGESTION

The following variables are used in this prompt:

- `state['previous_analysis_goals']`: Previous analysis goals from upstream visualization nodes.
- `state['data_summary']`: Data summary generated by Data Summarizer.

```
You are an experienced data analyst skilled in generating
analysis suggestions about data.

# Task
Generate 3-5 relevant next analysis steps as visualization
instructions.
Each suggestion should be a clear, actionable prompt for
creating a visualization that:
- Builds upon or complements the previous analysis goals
- Explores new aspects not yet covered
- Provides deeper insights or alternative perspectives

# Input
- Previous Analysis Goals:
{json.dumps(state['previous_analysis_goals'], indent=2)}
- Data Summary: {json.dumps(state['data_summary'], indent=2)}

# Output
Return ONLY a list of 3-5 suggestion strings.
Do not include any explanation, markdown formatting, or code
blocks. Just return the raw list.

# Guidelines
- DO NOT repeat analysis goals that have already been explored
- Suggest analyses that naturally follow from the previous
analysis goals
- Consider both broader explorations and deeper dives
- Mix different visualization types and analysis methods
```

C JSON SCHEMAS AND GRAPH PROPAGATION LOGIC

Each node and edge within the VISCANVAS graph is represented as a JSON object. These schemas define critical properties, including node positioning, the Vega-Lite specification, the LLM-generated rationale, the analysis goal, and topological connections.

The following is an example JSON object representing a node:

```
{
  "id": "ns4oNmeXUob3s0oKTKLch",
  "data": {
    "title": "Visualization",
    "status": "success",
    "icon": "BarChart",
    "visSpec": {
      "$schema":
```

```
"https://vega.github.io/schema/vega-lite/v6.json",
    "title": "Annual Distribution of Top Spotify Tracks",
    "description": "Bar chart showing the number of top
    Spotify tracks released each year from 1998 to 2020.",
    "data": {
      "url": "/data/songs_small.csv"
    },
    "transform": [
      {
        "calculate": "datum.year",
        "as": "release_year"
      },
      {
        "aggregate": [
          {
            "op": "count",
            "as": "song_count"
          }
        ],
        "groupby": ["release_year"]
      }
    ],
    "mark": {
      "type": "bar",
      "tooltip": true
    },
    "encoding": {
      "x": {
        "field": "release_year",
        "type": "ordinal",
        "title": "Year"
      },
      "y": {
        "field": "song_count",
        "type": "quantitative",
        "title": "Number of Songs"
      }
    }
  },
  "analysisGoal": "Determine the annual distribution of top
  Spotify tracks by counting the number of songs released each
  year from 1998 to 2020.",
  "rationale": "To visualize the distribution of top Spotify
  tracks by year, a bar chart is most appropriate as it clearly
  shows the count of songs released each year. The data needs to
  be aggregated by year, counting the number of songs per year.
  The x-axis will represent the year, and the y-axis will
  represent the count of songs. Since the dataset spans from 1998
  to 2020, the x-axis should be treated as a temporal or ordinal
  scale to accurately reflect the timeline. The bar height will
  correspond to the number of songs released in each year,
  providing a clear view of trends over time."
},
  "width": 549,
  "height": 457,
  "position": {
    "x": 470.5,
    "y": 40
  },
  "type": "generate-vis-node",
  "measured": {
    "width": 549,
    "height": 457
  },
  "selected": false
}
```

The following is an example JSON object representing an edge:

```
{
  "id": "ns4oNmeXUob3s0oKTKLch-AgNJTbtq-098DHm-tzbGj",
  "source": "ns4oNmeXUob3s0oKTKLch",
  "target": "AgNJTbtq-098DHm-tzbGj",
  "sourceHandle": "vis-generate-output",
  "targetHandle": "vis-placeholder-input",
```

```
"type": "placeholder-edge"  
}
```

The system supports reactive updates: re-running a *Text Input Node* automatically propagates new LLM-generated specifications to all connected *Visualization Nodes*. This chain-like propagation allows for the simultaneous revision of downstream leaf nodes. In the event of node deletion, the graph maintains continuity by reconnecting descendants to the deleted node's parent. However, because *Text Input Nodes* cannot be directly linked to one another, any orphaned *Text Input Nodes* resulting from a deletion are automatically removed.

D BACKEND SYSTEM FLOW

Figure 11 provides an overview of the backend workflow, including visualization specification generation, analysis goal suggestion, and the LLM-based generation pipeline.

E PARTICIPANT DEMOGRAPHICS

Table 1 summarizes participants' demographic information and prior experience with LLMs, visualization, data analysis, online whiteboards, and visualization libraries.

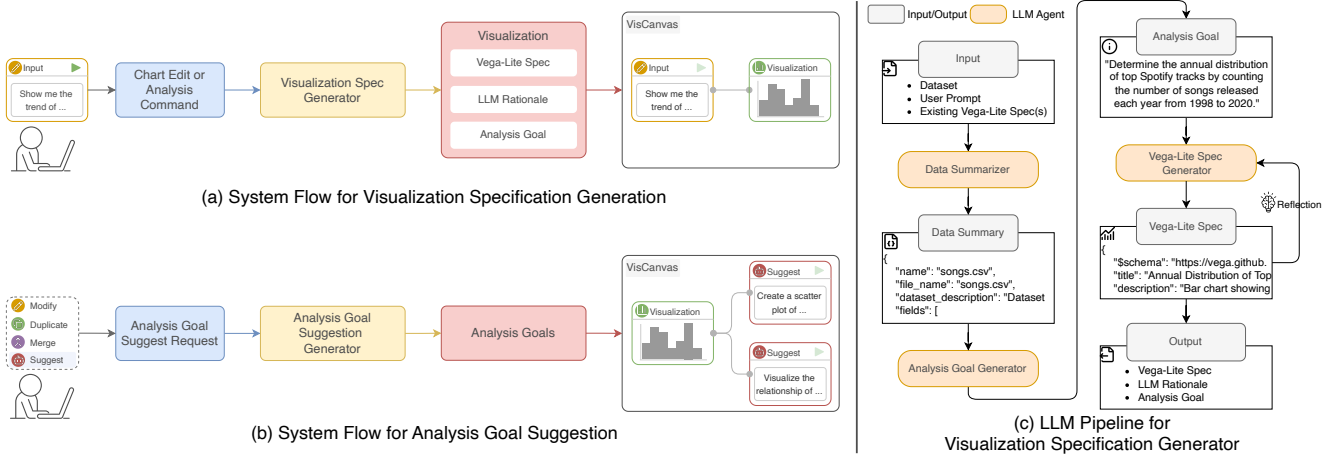


Figure 11: Backend system flow: (a) visualization specification generation; (b) analysis goal suggestion; (c) LLM pipeline for Visualization specification generator.

Table 1: Participant Demographics.

ID	Age	Gender	Frequency					Online Whiteboard tools	Vis Libraries
			LLM	LLM for Vis ²	Vis	Data Analysis			
P1	24	Female	5	3	4	4		2	D3, ggplot2, Matplotlib
P2	25	Male	5	2	2	2		2	D3
P3	25	Male	5	4	4	4		3	D3, Excel, Matplotlib, Tableau
P4	22	Male	5	2	3	2		—	D3, Excel, Matplotlib
P5	21	Male	5	2	5	4		2	D3, Excel, Matplotlib, PowerBI, Seaborn
P6	22	Female	5	4	4	4		3	D3, Excel, Matplotlib, Seaborn
P7	23	Male	5	5	5	5		1	D3, Excel, Matplotlib, PowerBI, Seaborn, Tableau
P8	22	Male	5	2	4	2		2	D3, Excel, Matplotlib
P9	25	Female	1	1	4	5		1	D3, Matplotlib, Seaborn
P10	26	Male	4	2	2	3		3	D3, Excel, Matplotlib, PowerBI
P11	23	Male	5	4	4	5		2	D3, ggplot2, Matplotlib
P12	22	Male	4	4	4	4		3	D3, Matplotlib, WandB
P13	25	Female	4	2	4	4		4	D3, Excel, Matplotlib, Seaborn
P14	23	Male	4	3	5	4		3	D3, Excel, Tableau
P15	23	Female	4	5	5	5		4	D3, Matplotlib, PowerBI, Tableau
P16	22	Female	5	4	4	5		4	D3, Matplotlib, PowerBI, Seaborn, Tableau
P17	25	Male	4	2	5	5		3	D3, Matplotlib, PowerBI, Seaborn
P18	22	Female	4	2	4	5		2	D3, Excel, Matplotlib
P19	21	Male	5	4	4	3		3	D3, Excel, Matplotlib, PowerBI
P20	24	Male	2	2	4	4		1	D3

¹Usage frequency scores: 5=At least once a day, 4=At least once a week, 3=At least once a month, 2=Less frequently but still occasionally, 1=Never. ²LLM usage for visualization authoring.

F PARTICIPANTS' EXPLORATION TOPOLOGIES

To code participants' exploration topologies, one author independently developed a codebook (reviewed and confirmed by the project team) using an open-coding method based on the taxonomy described in Sec. 5.2.4. Three authors then coded the entire topology dataset, achieving an overall initial agreement of 82.5%. The per-category agreements were: Linear (89%), Linear Parallel (90%), Tree (100%), Tree Parallel (100%), and Hybrid (62%). The *Hybrid* pattern was the most ambiguous class, with the lowest initial agreement, but all disagreements were resolved through discussion among the coders until consensus was reached across all topologies.

Exploration Topology by Interface

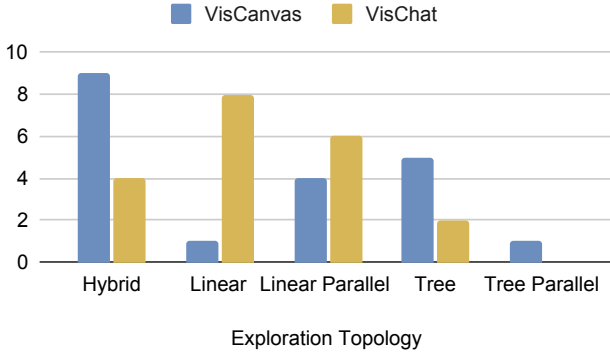


Figure 12: Distribution of participant topologies by interface. While VISCHAT performs comparably in linear exploration, VISCANVAS predominantly supports more divergent and complex behaviors, particularly in Hybrid and Tree Parallel structures.

The following figures illustrate individual participant topologies. Node colors and icons correspond to the node type:

-  **Text (Modify)** Text Input Nodes, where users issue natural-language instructions to generate or revise visualizations.
-  **Visualization** Visualization Nodes that render Vega-Lite charts generated from user instructions.
-  **Menu** Menu Nodes that expose four operations: *Modify*, *Duplicate*, *Merge*, and *Suggest*.
-  **Duplicate** Duplicated Visualization Nodes that copy a parent visualization.
-  **Merge** Merge Nodes that combine two Visualization Nodes into a single comparative visualization.
-  **Suggest** Suggested Text Input Nodes (3–5 candidates) generated from accumulated analysis goals.
-  **Fill Blank** Intermediate Visualization Nodes inserted between two connected Visualization Nodes.

Note: Visualization icons encircled by a magenta circle (●) indicate nodes that the participant bookmarked.

